

Module 3 React Fundamentals-2

PROF. SUNITA D RATHOD

REACT RENDERING HTML

React renders HTML to the web page by using a function called `createRoot()` and its method `render()`.

The `createRoot` Function

The `createRoot()` function takes one argument, an HTML element.

The purpose of the function is to define the HTML element where a React component should be displayed.

REACT RENDERING HTML

The render Method

The `render()` method is then called to define the React component that should be rendered.

But render where?

There is another folder in the root directory of your React project, named "public". In this folder, there is an `index.html` file.

You'll notice a single `<div>` in the body of this file. This is where our React application will be rendered.

Example

Display a paragraph inside an element with the id of "root":

```
const container = document.getElementById('root');  
const root = ReactDOM.createRoot(container);  
root.render(<p>Hello</p>);
```

The result is displayed in the `<div id="root">` element:

[index.html](#)

```
<body>  
  <div id="root"></div>  
</body>
```

The HTML Code

The HTML code in this uses JSX which allows you to write HTML tags inside the JavaScript code:

```
const myelement = (  
  <table>  
    <tr>  
      <th>Name</th>  
    </tr>  
    <tr>  
      <td>John</td>  
    </tr>  
    <tr>  
      <td>Elsa</td>  
    </tr>  
  </table>  
)
```

```
const container = document.getElementById('root');  
const root = ReactDOM.createRoot(container);  
root.render(myelement);
```

Example

The root node can be called whatever you like:

```
<body>  
  
  <header id="sandy"></header>  
  
</body>
```

Display the result in the `<header id="sandy">` element:

```
const container = document.getElementById('sandy');  
const root = ReactDOM.createRoot(container);  
root.render(<p>Hallo</p>);
```

What is JSX?

JSX stands for JavaScript XML.

JSX allows us to write HTML in React.

JSX makes it easier to write and add HTML in React.

Using JSX

```
const myElement = <h1>I Like JSX!</h1>;
```

```
const root =
```

```
ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(myElement);
```

Without using JSX

```
const myElement = React.createElement('h1', {}, 'I do not use JSX!');  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(myElement);
```

Expressions in JSX

With JSX you can write expressions inside curly braces `{ }`

The expression can be a React variable, or property, or any other valid JavaScript expression. JSX will execute the expression and return the result:

Example

Execute the expression `5 + 5`:

```
const myElement = <h1>React is {5 + 5} times better with JSX</h1>;
```

FRAGMENT

- you can use a "fragment" to wrap multiple lines.
- This will prevent unnecessarily adding extra nodes to the DOM.
- A fragment looks like an empty HTML tag: `<></>`.

Example

Wrap two paragraphs inside a fragment:

```
const myElement = (  
  <>  
    <p>I am a paragraph.</p>  
    <p>I am a paragraph too.</p>  
  </>  
);
```

Elements Must be Closed

JSX follows XML rules, and therefore HTML elements must be properly closed.

Example:

Close empty elements with `</>`

```
const myElement = <input type="text" />;
```

Attribute class = className

The `class` attribute is a much used attribute in HTML, but since JSX is rendered as JavaScript, and the `class` keyword is a reserved word in JavaScript, you are not allowed to use it in JSX.

Use attribute `className` instead.

Example:

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
  
const myElement = <h1 className="myclass">Hello World</h1>;  
  
const root =  
ReactDOM.createRoot(document.getElementById('root'));  
root.render(myElement);
```

Hello World

Conditions - if statements

React supports **if** statements, but not *inside* JSX.

To be able to use conditional statements in JSX, you should put the **if** statements outside of the JSX, or you could use a ternary expression instead

Example

Write "Hello" if x is less than 10, otherwise "Goodbye":

```
const x = 5;
let text = "Goodbye";
if (x < 10) {
  text = "Hello";
}
```

```
const myElement = <h1>{text}</h1>;
```

```
const x = 5;
```

```
const myElement = <h1>{(x) < 10 ? "Hello" : "Goodbye"}</h1>;
```

React Components

Components are independent and reusable bits of code. They serve the same purpose as JavaScript functions, but work in isolation and return HTML.

Components come in two types, Class components and Function components.

Function Component

A Function component also returns HTML, and behaves much the same way as a Class component, but Function components can be written using much less code, are easier to understand

```
function Car() {  
  return <h2>Hi, I am a Car!</h2>;  
}
```

EXAMPLE

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
  
function Car() {  
  return <h2>Hi, I am a Car!</h2>;  
}  
  
const root =  
ReactDOM.createRoot(document.getElementById('root')) ;  
  
root.render(<Car />) ;
```

OUTPUT:

Hi, I am a Car!

Class Component

A class component must include the **`extends React.Component`** statement.

This statement creates an inheritance to `React.Component`, and gives your component access to `React.Component`'s functions.

The component also requires a **`render()`** method, this method returns HTML.

Example

Create a Class component called `Car`

```
class Car extends React.Component {  
  render() {  
    return <h2>Hi, I am a Car!</h2>;  
  }  
}
```

Component Constructor

If there is a `constructor()` function in your component, this function will be called when the component gets initiated.

The constructor function is where you initiate the component's properties.

In React, component properties should be kept in an object called `state`.

Example;

```
class Car extends React.Component {  
  constructor() {  
    super();  
    this.state = {color: "red"};  
  }  
  render() {  
    return <h2>I am a {this.state.color} Car!</h2>;  
  }  
}  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Car />);
```

Props

Components can be passed as **props**, which stands for properties.

Props are like function arguments, and you send them into the component as attributes.

Example

```
function Car(props) {  
  return <h2>I am a {props.color} Car!</h2>;  
}  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Car color="red"/>);
```

Components in Components

```
function Car() {  
  return <h2>I am a Car!</h2>;  
}
```

```
function Garage() {  
  return (  
    <>  
      <h1>Who lives in my Garage?</h1>  
      <Car />  
    </>  
  );  
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Garage />);
```

OUTPUT:

Who lives in my Garage?

I am a Car!

Components in Files

React is all about reusing code, and it is recommended to split your components into separate files.

To do that, create a new file with a `.js` file extension and put the code inside it:

Note that the filename must start with an uppercase character.

EXAMPLE

This is the new file, we named it "Car.js":

```
function Car() {  
  return <h2>Hi, I am a Car!</h2>;  
}  
  
export default Car;
```

EXAMPLE

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import Car from './Car.js';  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Car />);
```

React Props

Props are arguments passed into React components.

Props are passed to components via HTML attributes.

props stands for properties.

Add a "brand" attribute to the Car element:

```
const myElement = <Car brand="Ford" />;
```

React Props

The component receives the argument as a **props** object:

```
function Car(props) {  
  return <h2>I am a { props.brand }!</h2>;  
}
```

React Props

Props are also how you pass data from one component to another, as parameters.

```
function Car(props) {  
  return <h2>I am a { props.brand }!</h2>;  
}
```

```
function Garage() {  
  return (  
    <>  
      <h1>Who lives in my garage?</h1>  
      <Car brand="Ford" />  
    </>  
  );  
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Garage />);
```

Props in the Constructor

If your component has a constructor function, the props should always be passed to the constructor and also to the `React.Component` via the `super()` method.

Props in the Constructor

```
class Car extends React.Component {  
  constructor(props) {  
    super(props);  
  }  
  render() {  
    return <h2>I am a {this.props.model}!</h2>;  
  }  
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Car model="Mustang"/>);
```

Creating the state Object

The state object is initialized in the constructor

The state object can contain as many properties as you like

```
class Car extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      brand: "Ford",  
      model: "Mustang",  
      color: "red",  
      year: 1964  
    };  
  }  
  render() {  
    return (  
      <div>  
        <h1>My Car</h1>  
      </div>  
    );  
  }  
}
```

Using the `state` Object

Refer to the `state` object anywhere in the component by using the `this.state.propertyname` syntax:

Example:

Refer to the `state` object in the `render()` method:

```
class Car extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      brand: "Ford",  
      model: "Mustang",  
      color: "red",  
      year: 1964  
    };  
  }  
}
```

```
render() {  
  return (  
    <div>  
      <h1>My {this.state.brand}</h1>  
      <p>  
        It is a {this.state.color}  
        {this.state.model}  
        from {this.state.year}.  
      </p>  
    </div>  
  );  
}
```

MCQ

Q - State whether true or false: React.js covers only the view layer of the app.

True

False

Q - The function which is called to render HTML to a web page in react?

ReactDOM_render()

render()

render_DOM()

DOM_HTML()

MCQ

Q - What are arbitrary inputs of components in react also known as?

Elements

Props

Keys

Ref

MCQ

What is ReactJS mainly used for building?

Database

Connectivity

User interface

Design platform

MCQ

Q - Which of the following are two ways to handle data in react?

services and components

State and props

state and services

state and component

Thank you